

NIT2112

Object Oriented Programming

Session 10

AI-Assisted Documenting

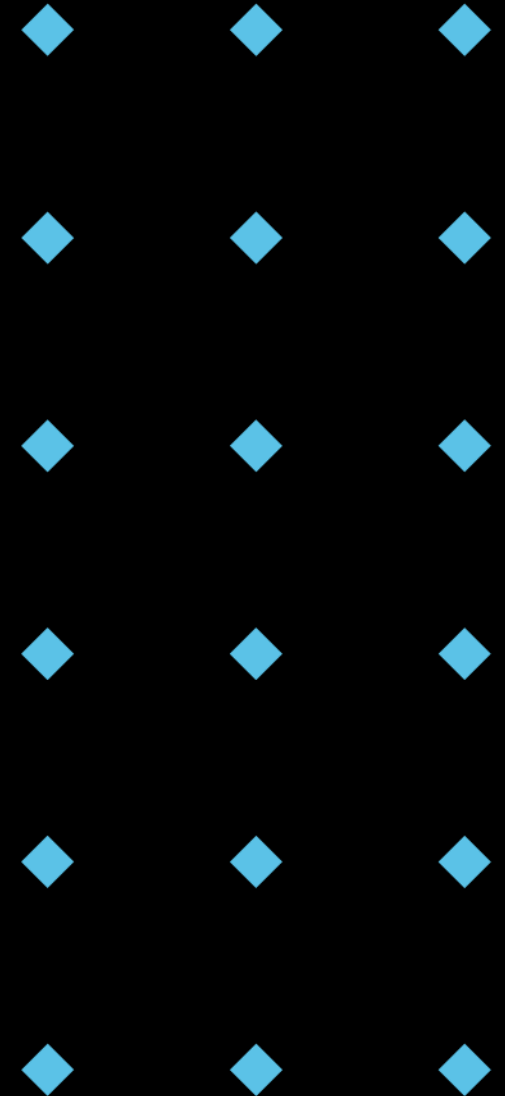
Acknowledgement of Country



Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- ◆ Importance & Types of Documentation
- ◆ AI for Generating Docstrings
- ◆ AI Assistance with Code Comments
- ◆ AI Support for UML Class Diagram
- ◆ AI for API Documentation & Developer Guides
- ◆ Workflow & Best Practices



Why Bother Documenting? Isn't Code Self-Explanatory?

- ◆ **Collaboration:** Enables team members (current and future) to understand and use your code.
- ◆ **Maintainability:** Makes it easier to debug, modify, and extend code later (even for yourself!). "Code tells you HOW, documentation tells you WHY."
- ◆ **Usability:** Helps others (or your future self) understand how to use your classes, methods, or APIs without reading every line of implementation.
- ◆ **Onboarding:** Speeds up the process for new developers joining a project.
- ◆ **Clarity of Thought:** The act of documenting can force you to clarify your own understanding of the code's purpose and design.
- ◆ **Reduces "Bus Factor":** Ensures project knowledge isn't lost if a key developer leaves.

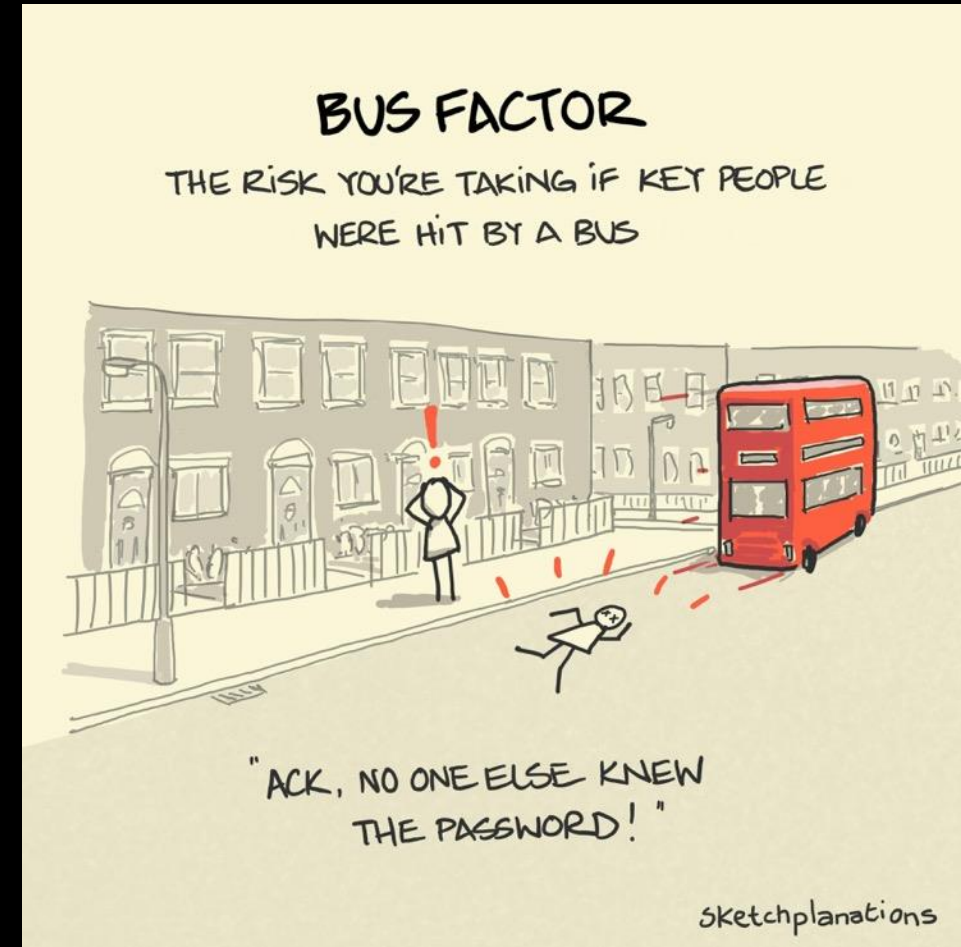
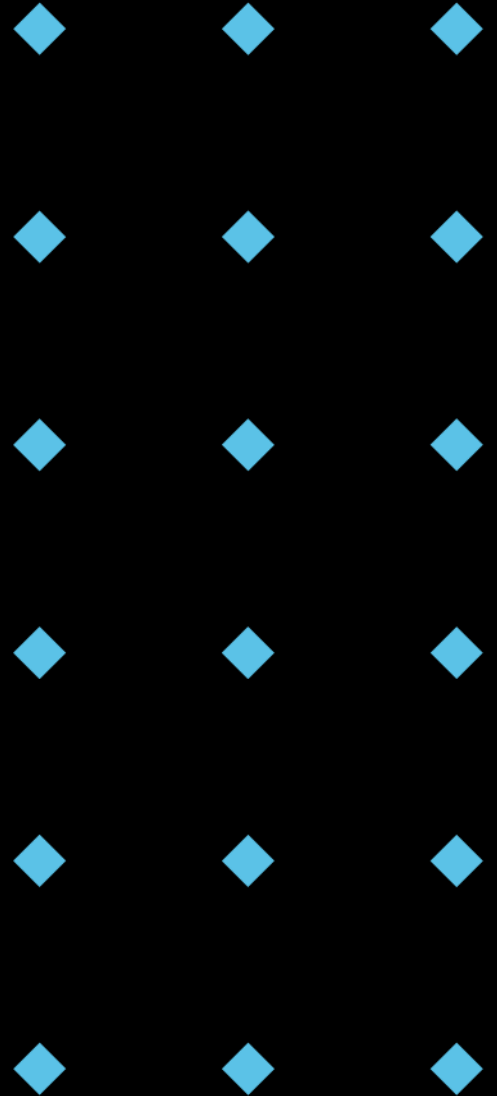


Image from Sketchplanations, 21 Nov 2022, <https://sketchplanations.com/bus-factor>

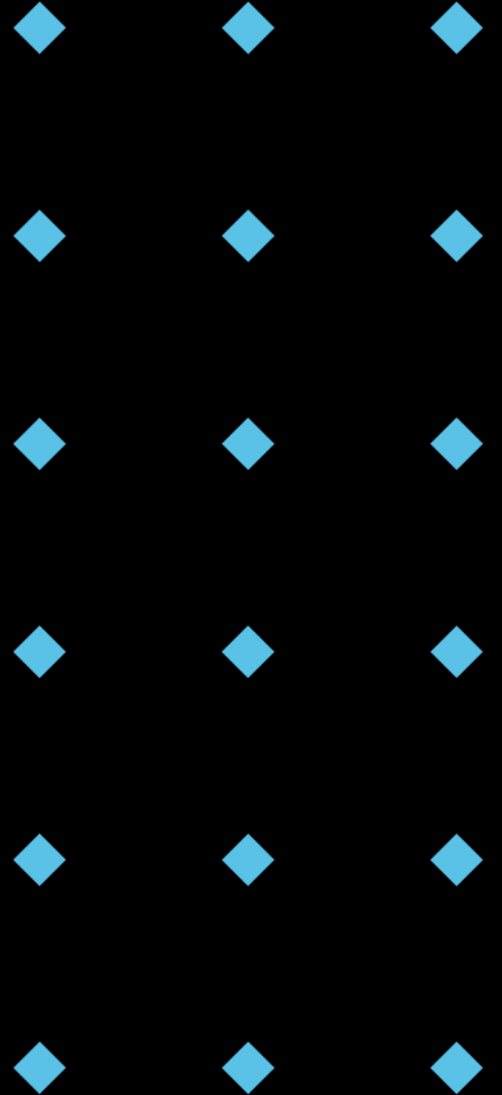
Different Levels, Different Purposes

- ◆ **Code Comments:** Inline explanations for specific lines or blocks of code (the 'how' or tricky 'whys').
- ◆ **Docstrings** (Documentation Strings): Describe modules, classes, functions, methods (the 'what' and 'how to use'). Standardized formats (e.g., PEP 257).
- ◆ **API Documentation:** For libraries or services. Details endpoints, parameters, return values, authentication, examples. Often generated from code/docstrings.
- ◆ **Tutorials & Guides:** Step-by-step instructions for common tasks or conceptual explanations.
- ◆ **Architecture Documentation:** High-level overview of the system's structure, components, and interactions (e.g., using UML diagrams).



Docstrings: Documenting Python Objects

- ◆ What: String literal occurring as the **first statement** in a module, function, class, or method definition.
- ◆ Purpose: Explains what the object does, its parameters, return values, raised exceptions, etc.
- ◆ Access: Available at runtime via the object's `__doc__` attribute.
- ◆ Tools: Used by `help()`, IDEs (for tooltips), and documentation generators (like Sphinx).



Docstrings: Documenting Python Objects

- ◆ Standard: PEP 257 provides conventions (not strict rules).
- ◆ There are four primary docstring formats: NumPy/SciPy docstrings, Google docstrings, reStructuredText, and Epytext.
- ◆ Key Elements (Common Styles):
 - ◆ One line Summary.
 - ◆ (Optional) More detailed explanation.
 - ◆ (Optional) Parameter descriptions.
 - ◆ (Optional) Return value description.
 - ◆ (Optional) Exceptions raised.

```
1 """
2 This is an example of Google style.
3
4 Args:
5     param1: This is the first param.
6     param2: This is a second param.
7
8 Returns:
9     This is a description of what is returned.
10
11 Raises:
12     KeyError: Raises an exception.
13 """
```

[Google Style](#)

```
1 """
2 This is a javadoc style.
3
4 @param param1: this is a first param
5 @param param2: this is a second param
6 @return: this is a description of what is returned
7 @raise keyError: raises an exception
8 """
```

[Epytext Style](#)

```
1 """
2 This is a reST style.
3
4 :param param1: this is a first param
5 :param param2: this is a second param
6 :returns: this is a description of what is returned
7 :raises keyError: raises an exception
8 """
```

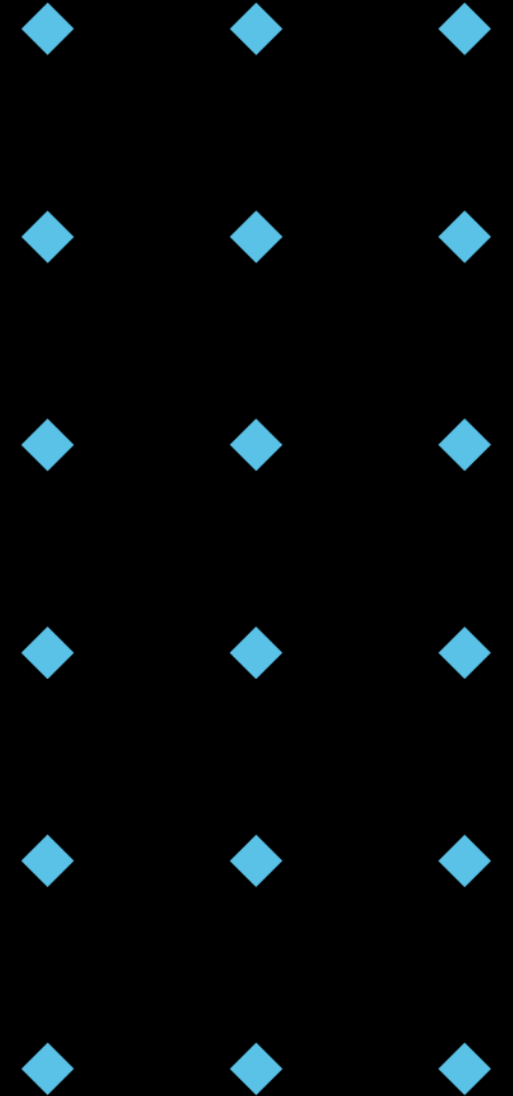
[reStructuredText Style](#)

```
1 """
2 My numpydoc description of a kind
3 of very exhaustive numpydoc format docstring.
4
5 Parameters
6 -----
7 first : array_like
8     the 1st param name `first`
9 second :
10     the 2nd param
11 third : {'value', 'other'}, optional
12     the 3rd param, by default 'value'
13
14 Returns
15 -----
16 string
17     a value in a string
18
19 Raises
20 -----
21 KeyError
22     when a key error
23 OtherError
24     when an other error
25 """
```

[Numpy Style](#)

AI Copilot to the Rescue: Generating Docstrings

- ◆ AI Copilots (e.g., GitHub Copilot) are trained on code and associated documentation (including docstrings).
- ◆ They can **analyse** your code (function signature, body, variable names) and **generate** appropriate docstrings.
- ◆ Often supports multiple common styles (reStructuredText, Google, NumPy, Epytext). Usually configurable in settings.
- ◆ Common Triggers:
 - ◆ Typing `"""` or `'''` after a function/class definition.
 - ◆ Using a specific command/shortcut (e.g., right-click -> "Copilot -> Generate Docs").
 - ◆ Using chat features: "Generate a docstring for the following function: [paste code]".



Example: AI Generating Docstrings for Rectangle

◆ Code before docstring

```
1 # Before Docstring
2 class Rectangle:
3     def __init__(self, width, height):
4         if width <= 0 or height <= 0:
5             raise ValueError("Dimensions must be positive")
6         self.width = width
7         self.height = height
8
9     def area(self):
10        return self.width * self.height
11
12    def perimeter(self):
13        return 2 * (self.width + self.height)
```

[Provide Code]

Generate a docstring for the class and functions above.

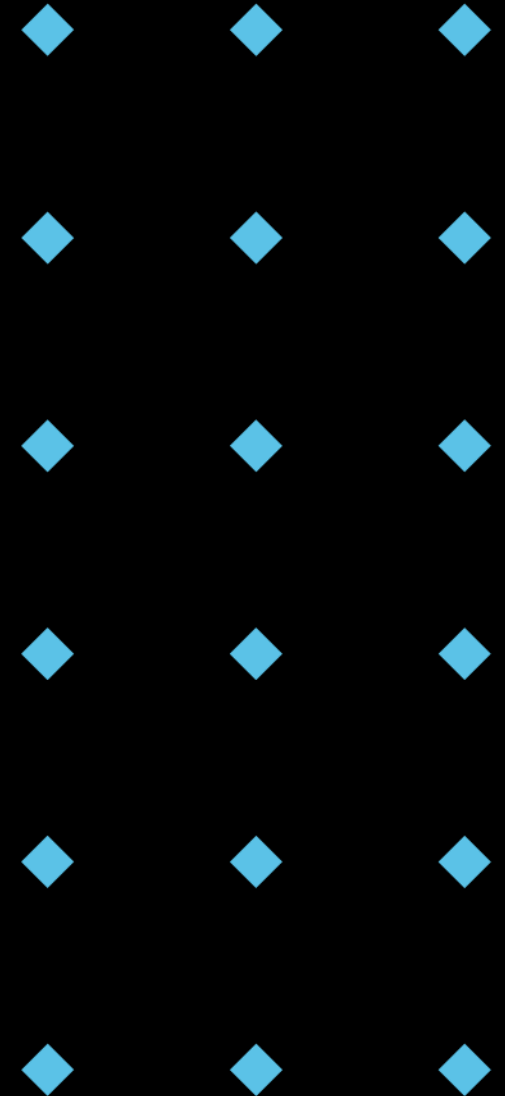
◆ Potential AI Output:



```
1 class Rectangle:
2     """
3     Represents a rectangle defined by width and height.
4
5     Attributes:
6         width (float): The width of the rectangle.
7         height (float): The height of the rectangle.
8     """
9     def __init__(self, width, height):
10        """
11        Initializes a new instance of the Rectangle class.
12
13        Args:
14            width (float): The width of the rectangle. Must be a positive
15                           number.
16            height (float): The height of the rectangle. Must be a positive
17                           number.
18
19        Raises:
20            ValueError: If width or height is less than or equal to 0.
21        """
22        if width <= 0 or height <= 0:
23            raise ValueError("Dimensions must be positive")
24        self.width = width
25        self.height = height
26
27    def area(self):
28        """
29        Calculates the area of the rectangle.
30
31        Returns:
32            float: The area of the rectangle (width * height).
33        """
34        return self.width * self.height
35
36    def perimeter(self):
37        """
38        Calculates the perimeter of the rectangle.
39
40        Returns:
41            float: The perimeter of the rectangle (2 * (width + height)).
42        """
43        return 2 * (self.width + self.height)
```

AI Isn't Magic: Review is Crucial

- ◆ **Accuracy:** AI might misunderstand complex logic or subtle nuances. It might **hallucinate** details or miss edge cases.
- ◆ **Context:** May lack the broader project context or design rationale (the "why").
- ◆ **Specificity:** Generated descriptions can sometimes be generic; they might need refinement for clarity.
- ◆ **Style Adherence:** Might not perfectly match project-specific style guide nuances unless carefully prompted or configured.
- ◆ **Over-Reliance:** Risk of accepting incorrect or poor documentation just because it was generated automatically.
- ◆ **Golden Rule:** **Always** review and edit AI-generated documentation. Treat it as a draft.



Discussion: Critique This AI Docstring

Questions:

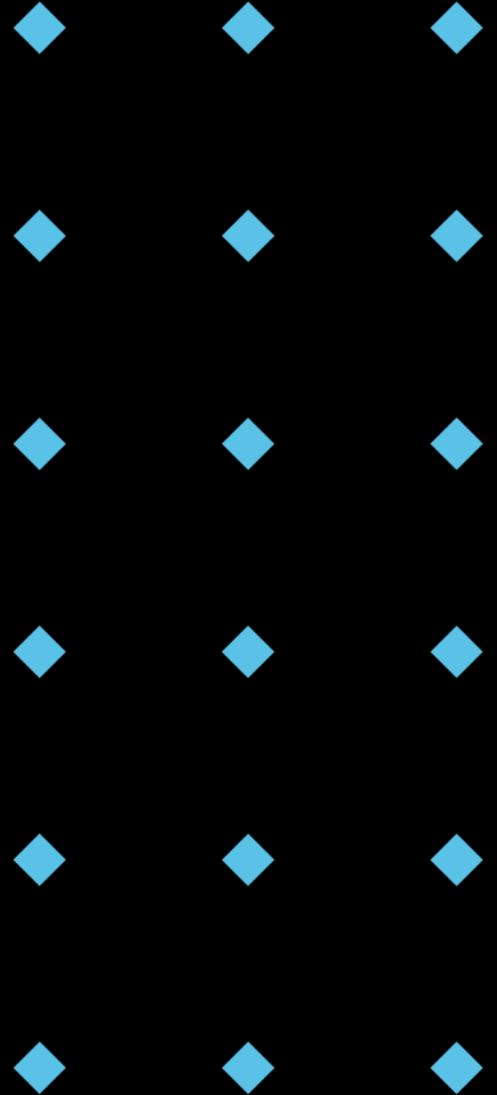
- ◆ Is this docstring helpful? Why or why not?
- ◆ What information is missing or unclear?
- ◆ How could you improve it (manually or by re-prompting the AI)?



```
1  def process_user_data(user_id, data, sensitive=False):
2      """
3      Processes user data.
4
5      Args:
6          user_id: The ID of the user.
7          data: The data to process.
8          sensitive: Whether the data is sensitive.
9
10     Returns:
11         The result of processing the data.
12     """
13     # ... complex logic here ...
14     if sensitive:
15         # ... special handling ...
16         result = f"Processed sensitive data for {user_id}"
17     else:
18         # ... standard handling ...
19         result = f"Processed data for {user_id}"
20     # ... more logic ...
21     return result # Simplified example return
```

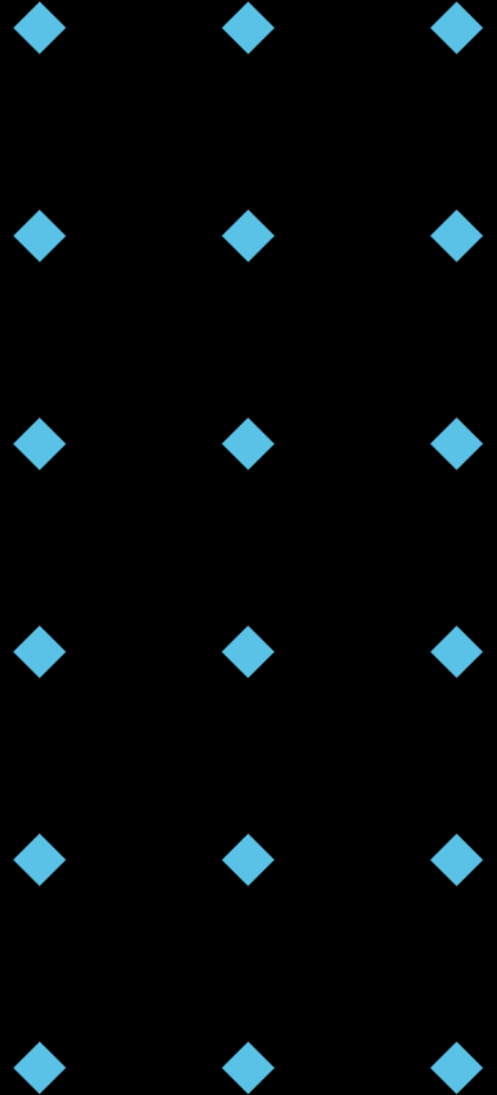
Beyond Docstrings: Code Comments

- ◆ Purpose: Explain why something is done **in a particular way**, clarify **complex** or **non-obvious** logic, mark **TODOs**, or provide **warnings**.
- ◆ Audience: Primarily **developers** maintaining the code.
- ◆ Placement: Inline (end of line) or on preceding lines.
- ◆ Key Difference from Docstrings: Comments explain **implementation details** or **rationale**; Docstrings explain **usage** and **purpose**.
- ◆ Good Commenting Principles:
 - ◆ Don't state the obvious (*`i = i + 1 # Increment i`*).
 - ◆ Explain the "why," not just the "what."
 - ◆ Keep comments up-to-date with code changes.
 - ◆ Use clear and concise language.



Can AI Help Write Good Comments?

- ◆ Yes, but with caution. AI Copilots can:
 - ◆ **Explain Code:** Select a block of code and ask the AI to explain it; use this explanation as a basis for a comment.
 - ◆ **Suggest Comments:** May automatically suggest comments for complex sections (less common than docstring generation).
 - ◆ **Refactor for Clarity:** Sometimes, the need for a comment indicates code could be clearer. AI can help refactor (e.g., extract method, rename variables), reducing the need for comments.
- ◆ Pitfalls:
 - ◆ AI might generate **obvious** or **unhelpful** comments.
 - ◆ AI might **misunderstand** the "why" just as easily as it misunderstands logic for docstrings.
 - ◆ **Over-commenting** can clutter code.
- ◆ Example Prompt:
 - ◆ "Explain this block of Python code and suggest a concise comment explaining its purpose: [paste code]"



Example: Using AI Explanation for Comments

◆ 1. Code to comment:

```
1 # Complex calculation (e.g., financial model, physics simulation)
2 magic_factor = 0.98
3 adjusted_value = (value * scale / offset) ** exponent
4 final_result = adjusted_value * magic_factor # Why 0.98?
```

[Provide Code]

Explain this block of Python code and suggest a concise comment explaining its purpose.

◆ 3. Code to comment:

The 0.98 factor represents a 2% reduction due to system friction, based on a specific model.

◆ 2. Potential AI Output:

This code calculates a final result based on an initial value, applying scaling, offset, and exponentiation. It then multiplies by a *magic_factor* of 0.98. This factor might represent an efficiency loss, a confidence adjustment, or a predefined constant from a specific model.

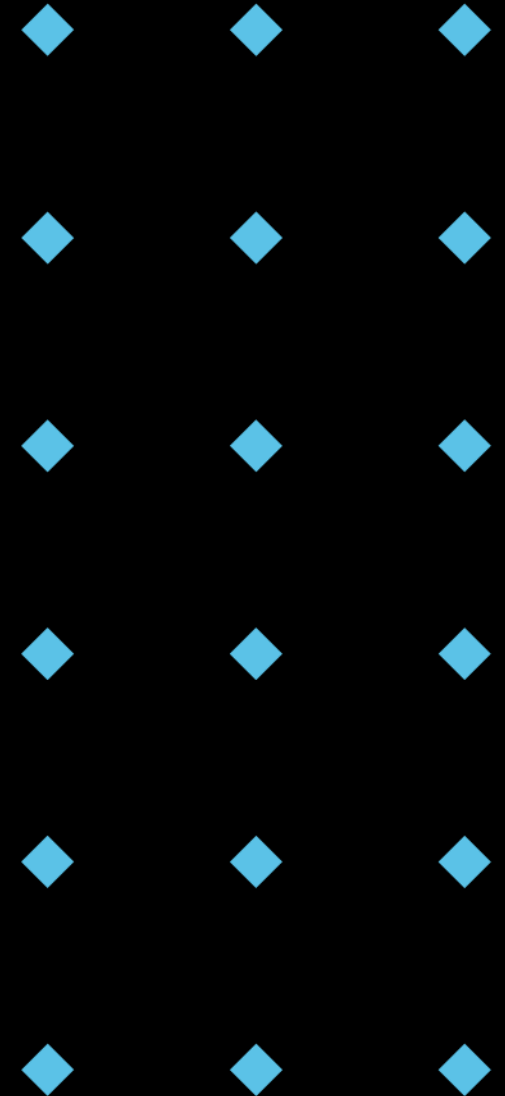
◆ 4. Potential AI Output:

```
1 # Apply scaling, offset, and exponent based on Model XYZ specification
2 adjusted_value = (value * scale / offset) ** exponent
3 # Apply 2% reduction (magic_factor=0.98) to account for observed system
  friction/loss
4 final_result = adjusted_value * magic_factor
```

Quick Quiz: Comment or Docstring?

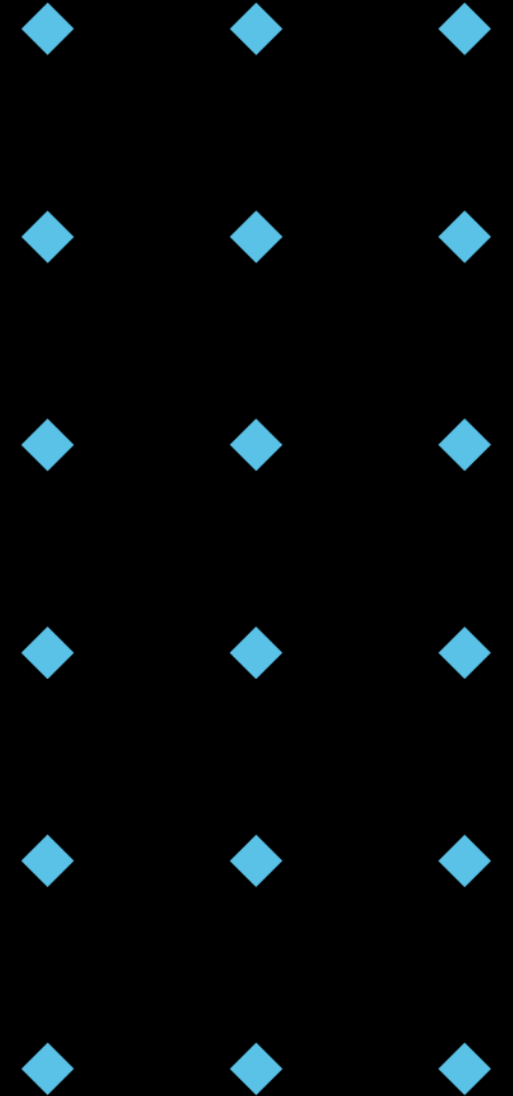
For each scenario, decide if a Docstring or a Code Comment is more appropriate:

- ◆ Explaining the expected dictionary structure for a function parameter config.
- ◆ Noting a temporary workaround for a known bug in an external library.
- ◆ Describing the overall purpose of a *UserProfile* class.
- ◆ Explaining why a bitwise shift ($\ll 1$) is used instead of multiplication by 2 for performance in a critical loop.
- ◆ Listing the possible exceptions a function might raise.



UML Diagrams for OOP

- ◆ UML (Unified Modeling Language): Standardized way to visualize the design of a system.
- ◆ Class Diagrams: A key type of UML diagram for OOP. Shows:
 - ◆ Classes
 - ◆ Attributes (variables)
 - ◆ Methods (functions)
 - ◆ Relationships between classes (Inheritance, Association, Aggregation, Composition).
- ◆ Benefits:
 - ◆ Provides a high-level overview of the system structure.
 - ◆ Helps understand relationships and dependencies.
 - ◆ Useful for design discussions and documentation.



AI Support for UML

- ◆ **PlantUML:** an open-source tool allowing users to create diagrams from a plain text language ("Diagrams as Code").
- ◆ **Explaining Diagrams:** You could potentially feed a generated diagram (or its textual representation like PlantUML code) to an AI and ask for explanations.
- ◆ **Generating PlantUML Code:** AI Copilots are often good at generating code in specific formats, including the text-based PlantUML syntax, based on your description or existing Python code.



Image from PlantUML Language Reference Guide (1.2025.0), 2025, <https://plantuml.com/guide>

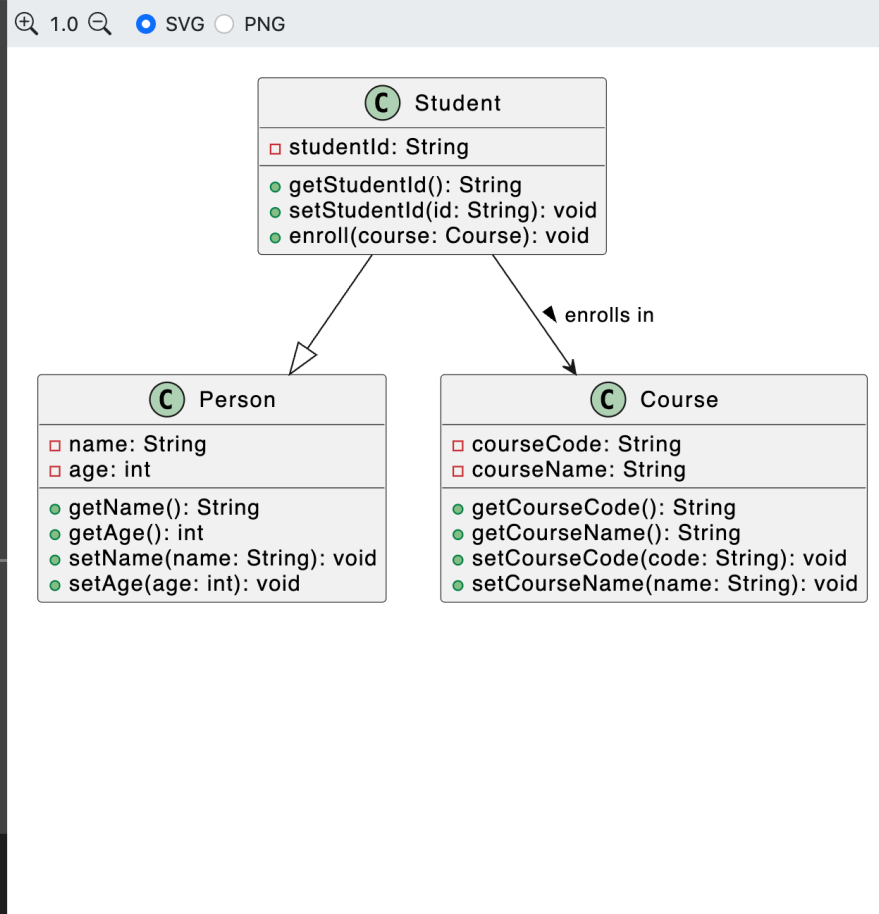
Example: Prompting AI for PlantUML Code

◆ 1. User Prompt:

[Provide Code]
Generate a PlantUML class diagram for the above Python classes.

◆ 2. Get potential AI output (PlantUML Syntax). Paste the code into a PlantUML renderer (online or local tool) to get the diagram image.

```
1 @startuml
2 ' Define Person class
3 class Person {
4     - name: String
5     - age: int
6     + getName(): String
7     + getAge(): int
8     + setName(name: String): void
9     + setAge(age: int): void
10 }
11
12 ' Define Student class that inherits from Person
13 class Student {
14     - studentId: String
15     + getStudentId(): String
16     + setStudentId(id: String): void
17     + enroll(course: Course): void
18 }
19
20 ' Define Course class
21 class Course {
22     - courseCode: String
23     - courseName: String
24     + getCourseCode(): String
25     + getCourseName(): String
26     + setCourseCode(code: String): void
27     + setCourseName(name: String): void
28 }
29
30 ' Inheritance relationship
31 Student --|> Person
32
33 ' Association relationship
34 Student --> Course : enrolls in >
35
36 @enduml
```



<https://plantuml-editor.kkeisuke.dev/>

Documenting Your API

- ◆ **API** (Application Programming Interface): A contract defining how different software components **interact**. In OOP, often refers to the public classes, methods, and functions of a **library** or **module**.
- ◆ Why Document APIs? Crucial for usability.
- ◆ Users need to know:
 - ◆ What functionality is available?
 - ◆ How to call functions/methods (parameters, types)?
 - ◆ What data structures are expected/returned?
 - ◆ What exceptions might be raised?
 - ◆ How to authenticate (if applicable)?
 - ◆ Example usage.
- ◆ Target Audience: **Developers** using your library, service, or module.

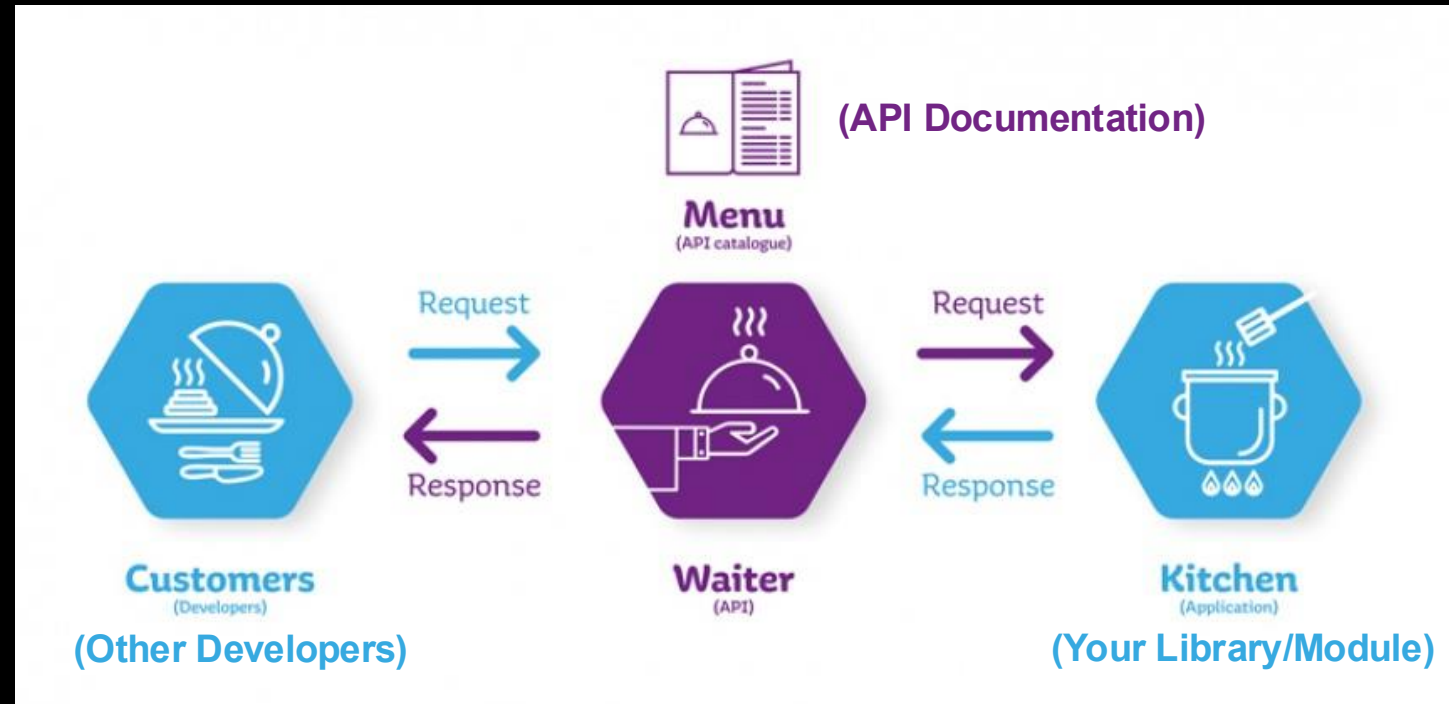


Image by Jeremy Mesiere from Manutan France, Sep 28, 2020,
<https://www.manutan.com/blog/en/glossary/api-definition-and-application-in-procurement>

Keys to Good API Documentation

- ◆ **Completeness:** Cover all public elements. Don't leave users guessing.
- ◆ **Accuracy:** Documentation **must** match the actual code behaviour. Keep it updated!
- ◆ **Clarity:** Use clear, unambiguous language. Define technical terms. **Structure:** Organize logically (e.g., by module, by functionality). Make it easy to navigate and search.
- ◆ **Consistency:** Use consistent terminology, formatting, and structure throughout.
- ◆ Examples: Provide practical, copy-paste-able code examples for common use cases.
- ◆ Tools: Leverage tools like Sphinx, ReadTheDocs, MkDocs which can often generate docs from your code's docstrings.



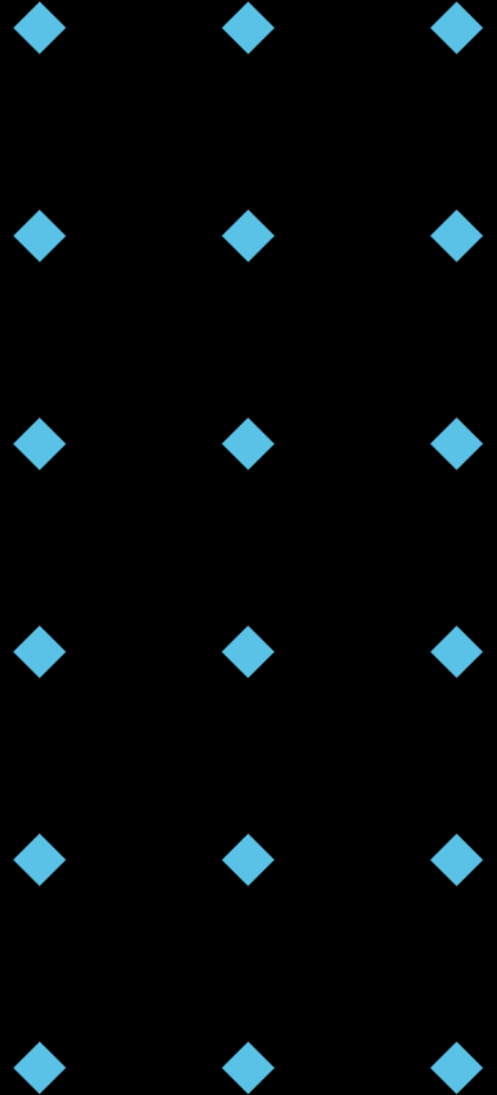
Sphinx
<https://www.sphinx-doc.org/en/master/>



Read the Docs
<https://about.readthedocs.com/>

How AI Can Help with API Docs

- ◆ **Generating Base Content from Docstrings:** Since tools like Sphinx use docstrings, AI's help in writing good docstrings directly improves generated API docs.
- ◆ **Writing Explanatory Text:** AI can help draft introductory paragraphs, explanations of concepts, or module overviews based on the code.
- ◆ **Generating Usage Examples:** Provide the AI with a function/method signature and ask it to generate realistic usage examples.
 - ◆ Prompt: "Generate a Python usage example for this function: ..."
- ◆ **Summarizing Code Functionality:** Ask AI to summarize what a module or complex class does, providing a basis for the API overview.
- ◆ **Ensuring Consistency:** AI can potentially help check for consistent terminology or formatting (though dedicated linters might be better).
- ◆ **Rephrasing/Clarification:** Use AI to rephrase complex technical descriptions into clearer language.



Visual: Example API Documentation

- ◆ An example API documentation generated using Sphinx, utilizing information from code docstrings and hosted on ReadTheDocs.

Requests

Requests is a simple, yet elegant, HTTP library.

```
>>> import requests
>>> r = requests.get('https://httpbin.org/basic-auth/user/pass', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.text
'{"authenticated": true, ...}'
>>> r.json()
{'authenticated': True, ...}
```

Requests allows you to send HTTP/1.1 requests extremely easily. There's no need to manually add query strings to your URLs, or to form-encode your `PUT` & `POST` data — but nowadays, just use the `json` method!

Requests is one of the most downloaded Python packages today, pulling in around 30M downloads / week — according to GitHub, Requests is currently [depended upon](#) by 1,000,000+ repositories. You may certainly put your trust in this code.

downloads/month

703M

python

3.8 | 3.9 | 3.10 | 3.11 | 3.12

contributors

403

Star

52,695

Requests is an elegant and simple HTTP library for Python, built for human beings. You are currently looking at the documentation of the development release.

Useful Links

[Quickstart](#)
[Advanced Usage](#)
[API Reference](#)
[Release History](#)
[Contributors Guide](#)

[Recommended Packages and Extensions](#)
[Requests @ GitHub](#)
[Requests @ PyPI](#)
[Issue Tracker](#)

Developer Interface

This part of the documentation covers all the interfaces of Requests. For parts where Requests depends on external libraries, we document the most important right here and provide links to the canonical documentation.

Main Interface

All of Requests' functionality can be accessed by these 7 methods. They all return an instance of the **Response** object.

`requests.request(method, url, **kwargs)` [\[source\]](#)

Constructs and sends a **Request**.

Parameters:

- **method** – method for the new **Request** object: GET, OPTIONS, HEAD, POST, PUT, PATCH, or DELETE.
- **url** – URL for the new **Request** object.
- **params** – (optional) Dictionary, list of tuples or bytes to send in the query string for the **Request**.
- **data** – (optional) Dictionary, list of tuples, bytes, or file-like object to send in the body of the **Request**.
- **json** – (optional) A JSON serializable Python object to send in the body of the **Request**.
- **headers** – (optional) Dictionary of HTTP Headers to send with the **Request**.
- **cookies** – (optional) Dict or CookieJar object to send with the **Request**.
- **files** – (optional) Dictionary of 'name': file-like-objects (or {'name': file-tuple}) for multipart encoding upload. file-tuple can be a 2-tuple ('filename', fileobj), 3-tuple ('filename', fileobj, 'content_type') or a 4-tuple ('filename', fileobj, 'content_type', custom_headers), where 'content_type' is a string defining the content type of the given file and custom_headers a dict-like object containing additional headers to add for the file.
- **auth** – (optional) Auth tuple to enable Basic/Digest/Custom HTTP Auth.
- **timeout** (*float or tuple*) – (optional) How many seconds to wait for the server to send data before giving up, as a float, or a ([connect timeout](#), [read timeout](#)) tuple.

Table of Contents

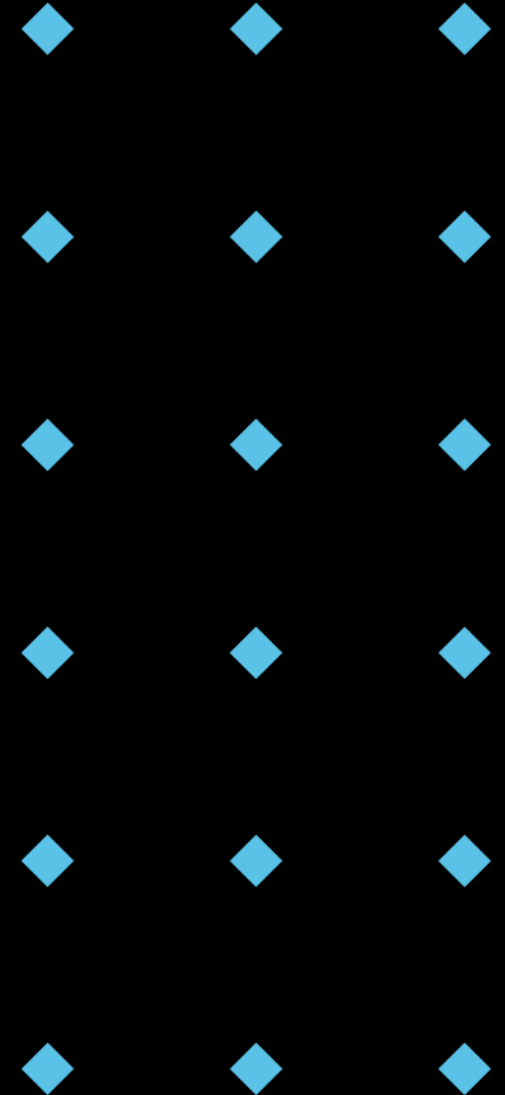
Developer Interface

- Main Interface
 - request
 - head
 - get
 - post
 - put
 - patch
 - delete
- Exceptions
 - RequestException
 - ConnectionError
 - HTTPError
 - TooManyRedirects
 - ConnectTimeout
 - ReadTimeout
 - Timeout
 - JSONDecodeError

Image from requests library, 2025, <https://github.com/psf/requests>

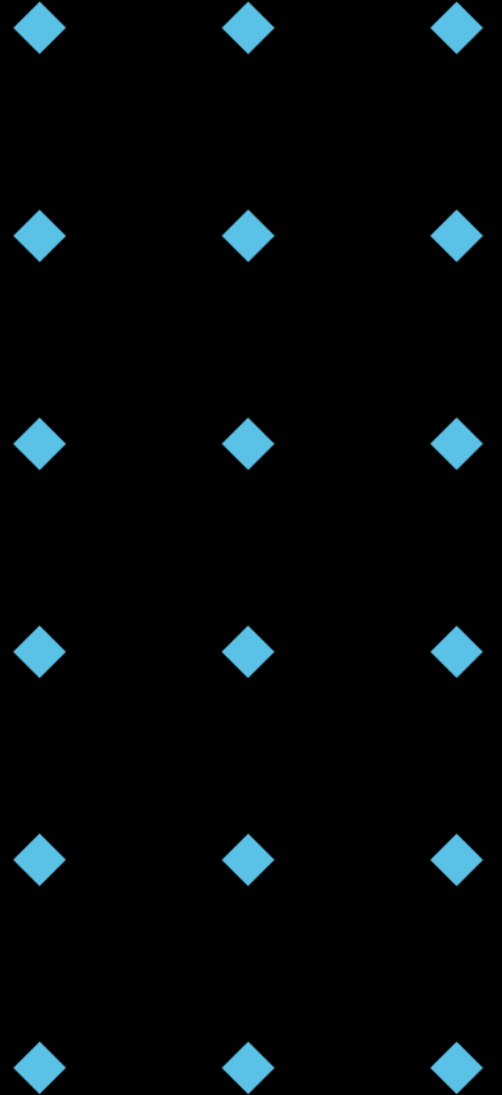
Refining Commenting Strategy with AI

- ◆ Leverage AI for **Explanations**: Use "Explain this code" frequently for complex sections. Decide if the explanation warrants a comment or if code refactoring (also AI-assisted) is better.
- ◆ Focus Comments on the **"Why"**: Let docstrings handle the "what." Use comments for non-obvious reasons, workarounds, or future intentions (TODO, FIXME). AI might guess the 'what' but rarely the 'why' without context.
- ◆ **No Boilerplate Comments**: If a comment feels like boilerplate (e.g., commenting every variable), the code or the comment is likely unnecessary. AI might offer these, but good judgment is needed.
- ◆ **Review AI Suggestions Critically**: Does the suggested comment add real value? Is it accurate? Is it concise? Does it explain the 'why'?
- ◆ Goal: Use AI to make commenting **more effective** and **less tedious**, not just to add more noise.



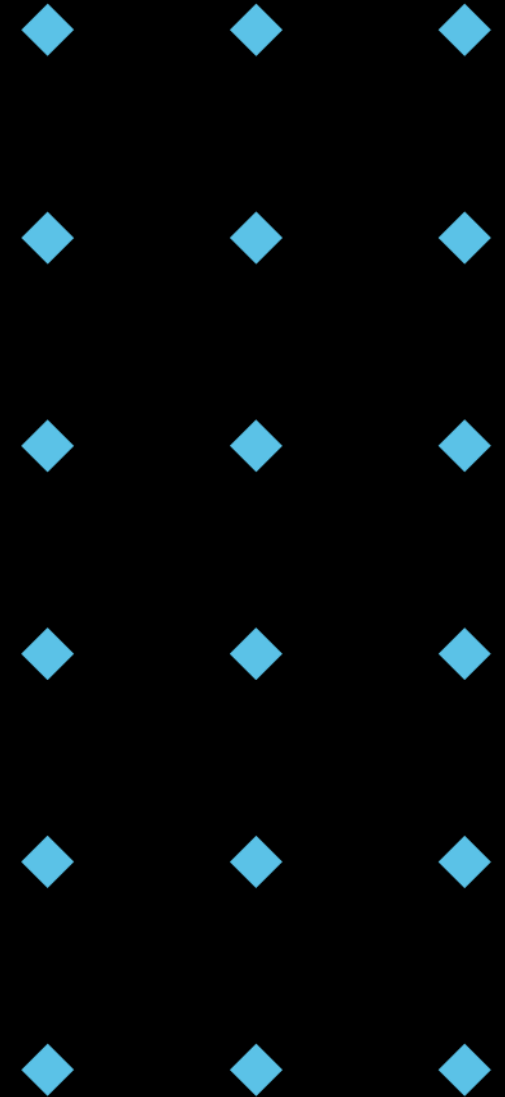
Beyond Reference: Guides & Tutorials

- ◆ Purpose: Help developers understand how to **achieve specific goals** using your library/system, or understand core concepts.
- ◆ Format: More narrative, **step-by-step** instructions, conceptual explanations.
- ◆ Examples:
 - ◆ "Getting Started Guide"
 - ◆ "How to Implement Feature X"
 - ◆ "Understanding the Authentication Flow"
 - ◆ "Advanced Usage Patterns"
- ◆ Difference from API Docs: API docs are a reference; guides/tutorials are **learning paths**.



AI for Crafting Guides & Tutorials

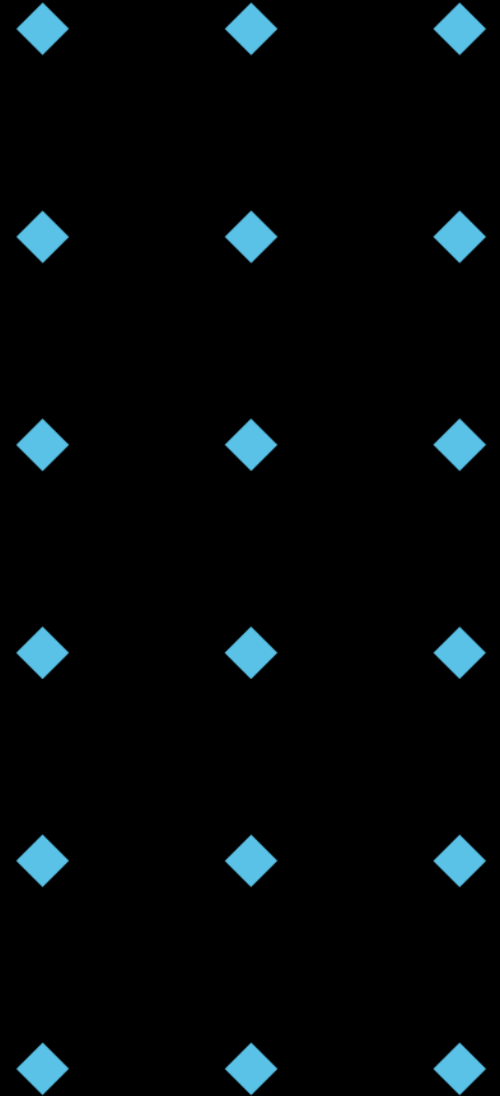
- ◆ **Outlining:** Ask AI to generate a potential outline for a tutorial on a specific topic (e.g., "Outline a tutorial for using the Rectangle class to calculate shapes in a drawing app").
- ◆ **Drafting Sections:** Provide AI with an outline point and relevant code snippets/API calls; ask it to draft an explanatory paragraph.
- ◆ **Generating Code Examples:** Create illustrative code snippets for tutorial steps.
- ◆ **Explaining Concepts:** Ask AI to explain a technical concept related to your library in simpler terms for the guide.
- ◆ **Rephrasing & Tone Adjustment:** Use AI to ensure the tone is appropriate for a tutorial (e.g., encouraging, clear) and to improve flow and readability.
- ◆ **Checking for Completeness:** Ask AI if the tutorial covers the necessary steps for a given task (based on the code/API).



Think-Pair-Share: Tutorial Topics

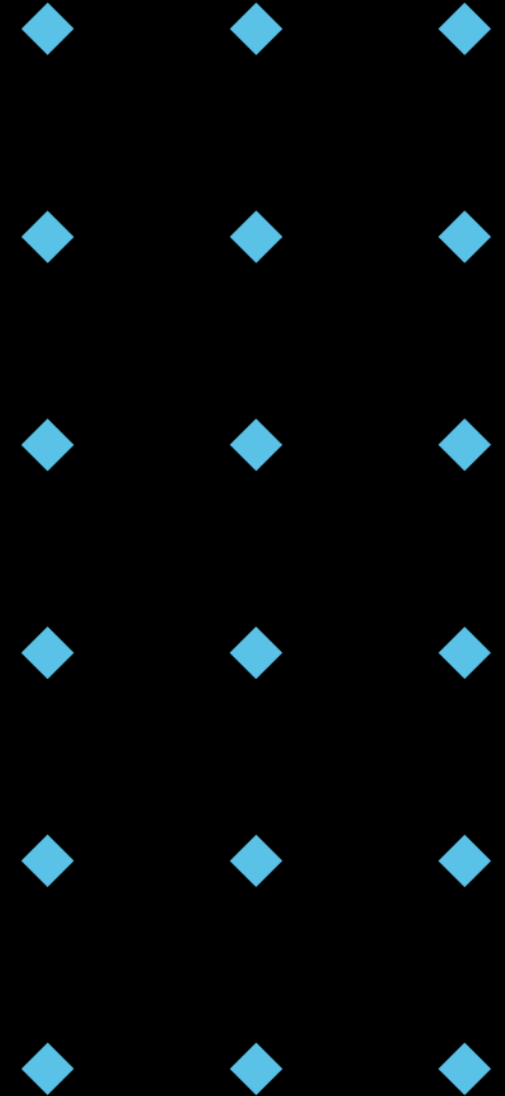
Scenario: Imagine we have developed a larger geometry library throughout this course (maybe including Circle, Triangle classes, etc.).

- ◆ Brainstorm 2-3 potential tutorial topics for this geometry library. What would users want to learn how to do?



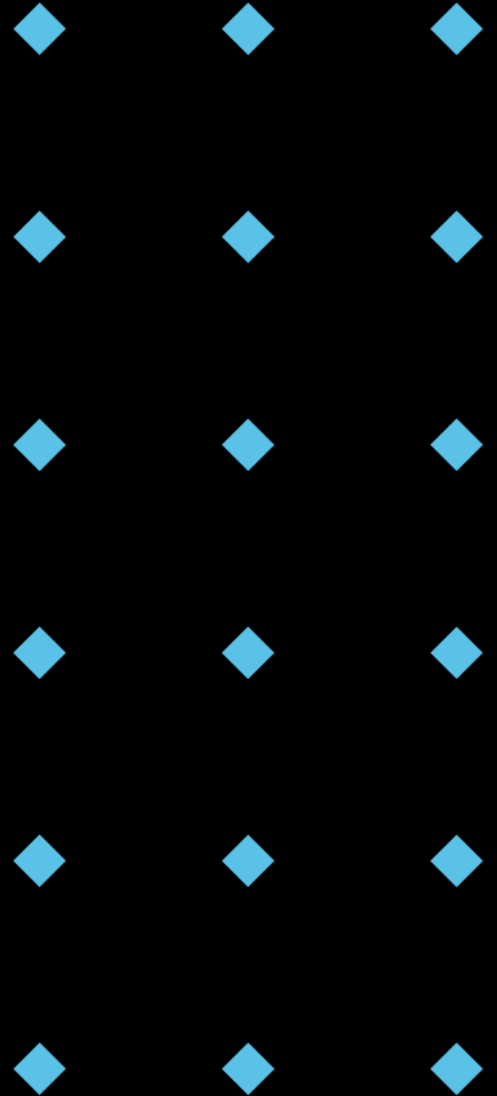
Putting It All Together: Workflow

- ◆ During Coding:
 - ◆ Use AI to generate initial docstrings for new classes/methods immediately after writing the code signature or body. Review & refine immediately.
 - ◆ Use AI "Explain Code" for complex logic; decide on comments vs. refactoring.
- ◆ Before Committing/Merging:
 - ◆ Review all generated docstrings and comments for accuracy and clarity.
 - ◆ Consider using AI to check for undocumented public methods/functions.
- ◆ Documentation Phase (for API docs/Guides):
 - ◆ Run documentation generator (e.g., Sphinx) based on docstrings.
 - ◆ Use AI to draft introductions, conceptual explanations, usage examples, and tutorial sections.
 - ◆ Use AI to generate PlantUML code for diagrams.
- ◆ Maintenance: When code changes, update docstrings/comments (AI can help redraft). Regenerate/update API docs and guides.



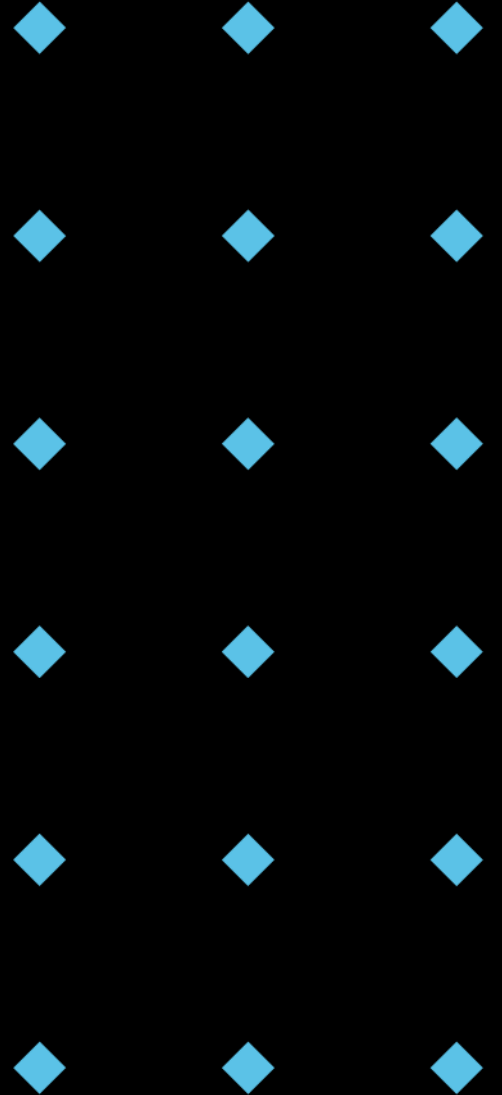
AI is a Tool, Not a Replacement

- ◆ **Critical Thinking:** AI doesn't understand intent or context like you do. You need to judge if the documentation is truly helpful, accurate, and complete.
- ◆ **Audience Awareness:** You understand your users (other developers, end-users) better than the AI. Tailor the language, examples, and depth accordingly.
- ◆ **Design Rationale:** The "why" behind design choices often needs human explanation. AI documents the "what" and "how."
- ◆ **Review and Curation:** The final responsibility for the quality and accuracy of documentation rests with the developer/team.
- ◆ **Consistency Across Project:** While AI helps with local consistency, ensuring overall narrative flow and conceptual integrity in guides requires human oversight.

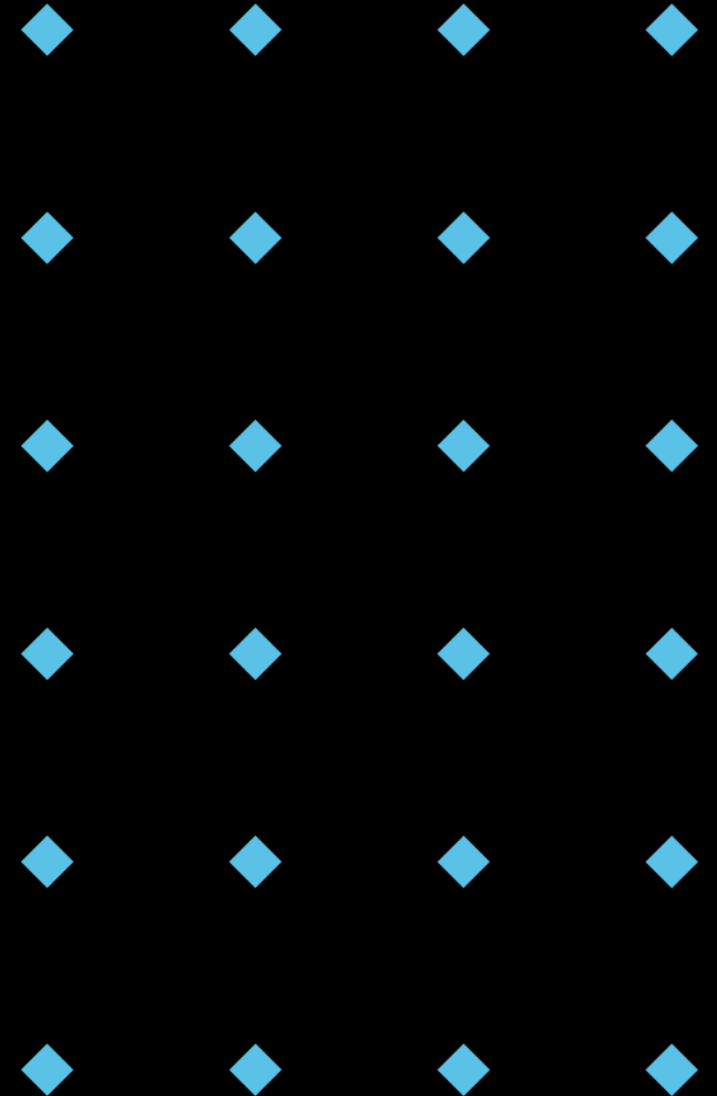


Key Takeaways

- ◆ Documentation is crucial for collaboration, maintainability, and usability.
- ◆ Various types exist: Comments, Docstrings, API Docs, Guides, UML, etc.
- ◆ AI Copilots excel at generating initial Docstrings based on code structure. Review is essential.
- ◆ AI can help explain code, aiding in writing better Code Comments (focus on 'why') or suggesting refactoring.
- ◆ AI can assist in generating code for UML diagrams (e.g., PlantUML) or explaining code structure.
- ◆ AI significantly speeds up creation of API Documentation and Guides/Tutorials by drafting text, generating examples, and outlining content.
- ◆ Integrate AI assistance into your workflow, but always maintain critical oversight and focus on clarity, accuracy, and audience needs.



Q & A



Lab

